

SirenOpt Multi-Sensor Data Acquisition — Design Note

1. Threading and Buffering

The system follows a producer–consumer architecture with three independent acquisition threads and a dedicated merge thread. Each sensor implements a common interface and submits readings to its own bounded, thread-safe FIFO queue (64 batches by default). Sensor A submits 1,000-sample blocks, while B and C submit individual readings.

Queue submission uses nonblocking *try_lock*. If a queue is full or its lock is unavailable, the incoming batch is dropped and counted. This drop-newest policy keeps acquisition independent of downstream processing while bounding memory usage.

The merge thread polls $A \rightarrow B \rightarrow C$, processing up to four complete batches per sensor per round. It inserts samples into a timestamp-ordered buffer and emits them after a 10 ms holdback to accommodate scheduling delays. Samples older than the last emitted timestamp are dropped; equal timestamps remain valid. The consumer processes samples continuously and reports statistics once per second. On shutdown, producers stop before the merge drains remaining data.

2. Timestamp Alignment

Each sensor has an independent clock, but only Sensor B provides an explicit device timestamp. Sensor A delivers callback blocks, while Sensor C provides untimestamped ASCII readings. Thus, the available information is insufficient to reconstruct a shared physical measurement timeline.

Using B's device clock as a reference would require a reliable mapping to the host clock and measurement-time information for A and C. UDP arrival timestamps alone cannot establish that mapping because they include variable network delay.

Instead, the system uses the host's monotonic clock as a common reference. Timestamps are assigned at callback observation (A), UDP reception (B), and complete-line reception (C). The merge orders samples by host observation time, preserving available device timing metadata separately. This provides consistent ordering without implying synchronization of physical measurements.

3. Sensor Failure Handling

Each sensor handles recoverable input failures independently, without stopping acquisition from the others.

- **Sensor A: Delayed callbacks;** Delayed 1,000-sample blocks may create bursts. The bounded queue absorbs temporary delays; rejected batches are dropped and counted. Callback observation times remain separate from simulated acquisition times.

- **Sensor B: UDP loss and reordering;** Packet validation and bounded sequence tracking detect duplicates, missing packets, out-of-order arrivals, and recovered gaps. The default `ArrivalOrder` mode forwards valid, non-late packets without waiting for missing sequences, while preserving sequence numbers and device timestamps.
- **Sensor C: Corrupted readings;** Malformed ASCII readings are rejected and counted; valid subsequent readings continue normally.

Shared drop accounting distinguishes queue/merge losses from invalid or duplicate readings rejected during acquisition. Final statistics reconcile received, emitted, and dropped samples.

4. Bottlenecks at 10x Rates

At 10x rates, Sensor A would produce 1,000,000 samples/s, B 10,000 packets/s, and C 100 readings/s. A would account for approximately 99% of the total.

The main scaling concern is the single-threaded merge stage. Approximately 10,100 samples would be generated during a 10 ms holdback interval, potentially exceeding its 8,192-sample buffer capacity. Individual insertions into a timestamp-ordered `std::map` also introduce ordering and allocation overhead, risking backlogs and drops.

Queue capacity and scheduling are additional concerns. Baseline experiments showed that polling fewer A batches reduced B latency but caused substantial A merge-late drops. We therefore retained 4/4/4 polling and selected the 10 ms holdback based on the observed latency–loss trade-off.

At 10x rates, I would first profile merge throughput, buffer occupancy, queue depths, polling delays, drops, and tail latency. Potential improvements include resizing buffers and reducing per-sample insertion overhead. These remain predicted bottlenecks, not verified 10x limits.

5. Why Sensor B Was Chosen

I chose to fully implement Sensor B (UDP) because it offers a combination of networking, data integrity, and timing challenges within a manageable scope. Unlike Sensor A's block-based acquisition or Sensor C's relatively straightforward serial parsing, Sensor B involves packet validation, loss, duplication, reordering, and independent device timestamps.

Though it may make the implementation more complex, these characteristics made Sensor B a good choice for demonstrating robust acquisition and failure handling. Its sequence numbers allow packet loss and reordering to be detected and tested, while device timestamps introduce meaningful questions about clock alignment and ordering across sensors. UDP also provides a practical way to simulate realistic failure conditions and validate the receiver end-to-end.